CORELINK — PLANO DE INCREMENTO FUNCIONAL • NATIVIDADE-TO

VULNERABILIDADE 1: SANITIZAÇÃO E INPUT INJECTION (CNPJ/CPF)

HJ Infor — Filtragem Estrita, Bloqueio de Payloads Maliciosos e Validação Algorítmica Módulo 11 no Perfil Corporativo

MÓDULO DE SEGURANÇA	CATEGORIA OWASP	STACK	DATA DE EXECUÇÃO
Perfil B2B & Server Actions	A03:2021 — Injection	Zod, Módulo 11, TypeScript	25/10/2026

1. VULNERABILIDADE SELECIONADA E PRIORIZAÇÃO

VULNERABILIDADE MITIGADA

Vulnerabilidade 1 (Sanitização e Input Injection): O backend aceitava strings sem filtragem prévia para CNPJ/CPF e campos correlatos no perfil corporativo, permitindo injeção de parâmetros maliciosos ou formatação inconsistente.

Justificativa Técnica: A ausência de sanitização prévia e de verificação algorítmica nos endpoints `updateUserProfile` (`src/actions/user.ts`) e `registerUser` (`src/actions/auth.ts`) permitia: (1) envio de vetores XSS/HTML (`<script>`), SQLi e injeção de parâmetros na integração WhatsApp API (UC05); (2) aprovação de CNPJs/CPFs forjados com sequências repetidas (`00.000.000/0000-00`) ou dígitos verificadores inválidos; e (3) gravação inconsistente no PostgreSQL (ora apenas números, ora com pontuação parcial).

2. RESULTADO ESPERADO E COMPORTAMENTO ALCANÇADO

VALIDAÇÃO MÓDULO 11 ATIVA

Foi construído o módulo `src/lib/sanitize.ts` aplicado no cliente (`ProfileForm.tsx`), nas Server Actions (`user.ts` e `auth.ts`) e no gerador WhatsApp (`whatsapp.ts`). Qualquer tentativa de injeção (`hasMaliciousInjection`) é imediatamente bloqueada e registrada em log. O campo CNPJ/CPF passa por extração numérica estrita (`onlyDigits`), cálculo dos **2 Dígitos Verificadores Módulo 11** (`isValidCPF` / `isValidCNPJ`) e normalização canônica obrigatória (`XX.XXX.XXX/XXXX-XX` ou `XXX.XXX.XXX-XX`) antes de gravar no Prisma ORM.

3. PLANO DE EXECUÇÃO DETALHADO

ATIVIDADE	RESPONSÁVEL	TECNOLOGIAS	PERÍODO	STATUS
Criação do Motor de Sanitização (`src/lib/sanitize.ts`). Implementação de `hasMaliciousInjection`, `sanitizeTextInput`, `isValidCPF`, `isValidCNPJ` e `validateAndSanitizeDocument`.	Marques Ramirez	TypeScript, RegExp Anti-XSS/SQLi, Algoritmo Módulo 11	22/10 a 23/10/2026	✓ Concluída
Blindagem das Server Actions (`user.ts` e `auth.ts`). Filtragem estrita pré-banco, validação Zod com `.transform()` e guarda RBAC em `updateUserRole`.	Márcio Medrado	Next.js Server Actions, Zod Schema, Prisma ORM	23/10 a 24/10/2026	✓ Concluída
Máscara Dinâmica e Validação Visual (`ProfileForm.tsx`). Formatação instantânea ao digitar e badge em tempo real de autenticidade Módulo 11.	Marques Ramirez & Márcio Medrado	React 19, Tailwind CSS, Sanitizer Client/Server	24/10 a 25/10/2026	✓ Concluída
Auditoria de Injeção e Testes de Penetração. Execução da bateria `scripts/test-sanitization-vuln1.mjs` contra vetores SQLi, XSS e CNPJs falsos.	Taís Fernandes, Romildo & Vandelice	Node.js, Vetores OWASP A03, Receita Federal DV	25/10/2026	✓ Concluída
Documentação de Segurança e Relatório PDF. Consolidação das evidências de código e logs de bloqueio no padrão de 2 páginas A4.	Andréa Ribeiro	Git, GitHub, Markdown, PDF Engine	25/10/2026	✓ Concluída

4. ESPECIFICAÇÃO DAS REGRAS DE SANITIZAÇÃO E NORMALIZAÇÃO IMPLEMENTADAS

FILTRO MULTICAMADA

FUNÇÃO / REGRA (`SANITIZE.TS`)	CAMPOS PROTEGIDOS	MECANISMO DE FILTRAGEM E PADRONIZAÇÃO NO POSTGRESQL
`validateAndSanitizeDocument` (`isValidCNPJ` / `isValidCPF`)	`document` (CNPJ ou CPF no Perfil Corporativo)	Bloqueia letras/símbolos maliciosos, rejeita sequências repetidas (`000...`), valida os 2 dígitos Módulo 11 e grava sempre formatado (`XX.XXX.XXX/XXXX-XX`).
`hasMaliciousInjection` & `sanitizeTextInput`	`name`, `companyName`, `address`	Detecta e rejeita `<script>`, tags HTML, `javascript:`, `on*=`, `DROP/UNION SELECT`, comentários SQL (`--`) e caracteres de controle.
`sanitizeStateRegistration`	`stateRegistration` (Inscrição Estadual)	Aceita exclusivamente a palavra reservada `"ISENTO"` ou entre 7 e 14 dígitos numéricos puros, eliminando injeção de parâmetros fiscais.
`sanitizeAndFormatPhone`	`phone` (WhatsApp B2B)	Filtra caracteres não-numéricos, exige DDD válido (10 ou 11 dígitos) e padroniza no formato `(XX) XXXXX-XXXX`.

5. EVIDÊNCIA VISUAL: FILTRO DE INJEÇÃO E VALIDAÇÃO MÓDULO 11 EM TEMPO REAL

AUDITORIA OWASP A03

PAYLOADS SQLI / XSS TESTADOS	ALGORITMO RECEITA FEDERAL	PADRONIZAÇÃO NO BANCO	CAMADAS DE VALIDAÇÃO
100% Bloqueados	Módulo 11 (2 DVs)	Máscara Canônica	UI + Zod + Action

1. Entradas Maliciosas ou Falsas (Rejeitadas pelo Backend)

Tentativas de Input Injection e CNPJs forjados interceptados

`11222333000181'; DROP TABLE User;--`	✗ Injection Bloqueado
`<script>fetch('evil.com')</script>`	✗ XSS Bloqueado
`00.000.000/0000-00 | 11.222.333/0001-99`	✗ DV Módulo 11 Inválido

2. Entradas Autênticas (Normalizadas no PostgreSQL)

Formatação automática independente da pontuação digitada

`Input: 11222333000181 ➔ 11.222.333/0001-81`	✓ CNPJ Válido (Mod 11)
`Input: 52998224725 ➔ 529.982.247-25`	✓ CPF Válido (Mod 11)
`Tel: 63999998888 ➔ (63) 99999-8888`	✓ Formato Canônico

6. IMPLEMENTAÇÃO TÉCNICA (SRC/LIB/SANITIZE.TS & SRC/ACTIONS/USER.TS) E LOG DE AUDITORIA

CÓDIGO & TELEMETRIA

6.1 Validação Algorítmica CNPJ e Anti-Injection (src/lib/sanitize.ts)

```
export function isValidCNPJ(raw: string | null | undefined): boolean {
  const digits = onlyDigits(raw);
  if (digits.length !== 14 || /^(\d)\1{13}$/.test(digits)) return false;

  const calcDigit = (slice: string, weights: number[]): number => {
    const sum = weights.reduce((acc, w, i) => acc + Number(slice[i]) * w, 0);
    const mod = sum % 11;
    return mod < 2 ? 0 : 11 - mod;
  };
  const w1 = [5, 4, 3, 2, 9, 8, 7, 6, 5, 4, 3, 2];
  const w2 = [6, 5, 4, 3, 2, 9, 8, 7, 6, 5, 4, 3, 2];

  return (
    calcDigit(digits.slice(0, 12), w1) === Number(digits[12]) &&
    calcDigit(digits.slice(0, 13), w2) === Number(digits[13])
  );
}

export function validateAndSanitizeDocument(raw: string | null | undefined) {
  if (hasMaliciousInjection(raw) || /[a-zA-Z<>;'"`\\]/.test(raw || "")) {
    return { valid: false, type: "INVALID", formatted: null,
      error: "Entrada bloqueada: tentativa de injeção detectada." };
  }
  const digits = onlyDigits(raw);
  if (digits.length === 14 && isValidCNPJ(digits))
    return { valid: true, type: "CNPJ", formatted: formatCpfCnpjMask(digits) };
  // ...
}
```

6.2 Log Real da Auditoria (scripts/test-sanitization-vuln1.mjs)

```
// src/actions/user.ts — Guarda Server-Side antes do Prisma
for (const field of rawFields) {
  if (hasMaliciousInjection(field.value)) {
    return {
      success: false,
      error: `Bloqueio de Segurança: "${field.label}" contém instruções proibidas.`,
    };
  }
}
const docCheck = validateAndSanitizeDocument(data.document);
if (!docCheck.valid) return { success: false, error: docCheck.error };

======================================================================
 AUDITORIA DE SEGURANCA - VULNERABILIDADE 1 (SANITIZACAO E INJECTION)
======================================================================
 [BLOQUEADO] SQL Injection no CNPJ    → BLOQUEADO (INJECTION)
 [BLOQUEADO] XSS Payload no Perfil    → BLOQUEADO (INJECTION)
 [BLOQUEADO] CNPJ Sequencia Falsa     → REJEITADO (DV CNPJ INVALIDO)
 [BLOQUEADO] CNPJ Digito Invalido     → REJEITADO (DV CNPJ INVALIDO)
 [ACEITO   ] CNPJ Sem Mascara Valido  → Gravado: 11.222.333/0001-81
 [ACEITO   ] CPF Sem Mascara Valido   → Gravado: 529.982.247-25
======================================================================
```

7. ROTEIRO DE VALIDAÇÃO E TESTES DE SEGURANÇA (HOMOLOGAÇÃO)

CENÁRIO DE TESTE	PROCEDIMENTO EXECUTADO	RESULTADO OBTIDO NA HOMOLOGAÇÃO
Injeção SQL / Script no Campo CNPJ/CPF	Submeter payload `11222333000181'; DROP TABLE User; --` em `updateUserProfile`.	A função `hasMaliciousInjection` interceptou o payload antes do banco e retornou bloqueio imediato de segurança.
Injeção XSS na Razão Social (`companyName`)	Inserir `<script>alert(1)</script>` no formulário de perfil corporativo.	Bloqueado tanto no cliente (`ProfileForm.tsx`) quanto na Server Action (`user.ts`) e sanitizado em `whatsapp.ts`.
CNPJ com Dígito Verificador Forjado	Informar `00.000.000/0000-00` ou `11.222.333/0001-99` no campo de documento.	O algoritmo Módulo 11 (`isValidCNPJ`) detectou divergência nos dígitos verificadores e impediu a gravação.
Normalização de CNPJ/CPF Sem Pontuação	Enviar `11222333000181` (apenas dígitos) para o backend.	Validado com sucesso e gravado no PostgreSQL no formato canônico padronizado `11.222.333/0001-81`.